Data Sciences – CentraleSupélec
Advanced Machine Learning
Course IV – Clustering

Emilie Chouzenoux

Center for Visual Computing
CentraleSupélec, Inria Saclay
emilie.chouzenoux@centralesupelec.fr

From observations to groups

$$\mathcal{D}_n = \{\mathbf{x}_i\}_{i=1}^n, \qquad \mathbf{x}_i \in \mathbb{R}^d.$$

No observed target labels

- Group observations using a chosen notion of similarity.
- Interpret the groups, summarize the data or build a representation.

From observations to groups

$$\mathcal{D}_n = \{\mathbf{x}_i\}_{i=1}^n, \qquad \mathbf{x}_i \in \mathbb{R}^d.$$

No observed target labels

- Group observations using a chosen notion of similarity.
- Interpret the groups, summarize the data or build a representation.

Data / features $\longrightarrow$ Dissimilarity

$\longrightarrow$ Criterion or construction

$\longrightarrow$ Algorithm $\longrightarrow$ Interpretation

Course I: separating the modeling choice from its computation.

What should a cluster represent?

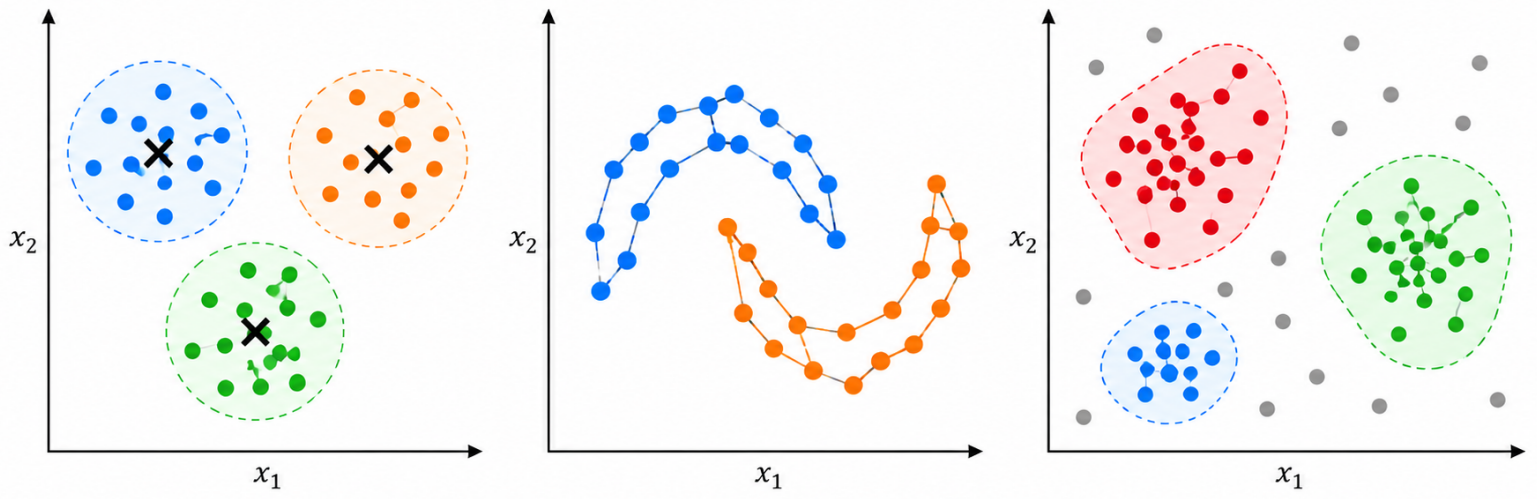
Different questions lead to different groups

- **Prototype:** observations close to a representative point.
- **Connectivity:** observations linked by short steps.
- **Density:** dense regions separated by sparse regions.

Application	Observations and possible interpretation
Patient grouping	Biological features; patient profiles
Image segmentation	Pixel features; appearance regions
Colour quantization	RGB values; representative colours

A cluster is meaningful relative to features and a scientific question.

What should a cluster represent?



Dissimilarity and metric distances

We group similar observations, but many clustering algorithms measure how *different* they are. A symmetric dissimilarity δ satisfies

$$\delta(\mathbf{u}, \mathbf{v}) \geq 0, \qquad \delta(\mathbf{u}, \mathbf{v}) = \delta(\mathbf{v}, \mathbf{u}), \qquad \delta(\mathbf{u}, \mathbf{u}) = 0.$$

A **metric** also satisfies

$$\delta(\mathbf{u}, \mathbf{v}) = 0 \iff \mathbf{u} = \mathbf{v}, \qquad \delta(\mathbf{u}, \mathbf{v}) \leq \delta(\mathbf{u}, \mathbf{w}) + \delta(\mathbf{w}, \mathbf{v}).$$

Example
Squared Euclidean dissimilarity $\delta(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2^2$ is not a metric:

$$|0 - 2|^2 = 4 > |0 - 1|^2 + |1 - 2|^2 = 2.$$

Some clustering algorithms require only pairwise dissimilarities, whereas others rely on Euclidean geometry.

Examples of metric distances

Minkowski, for $q \geq 1$:

$$\delta_q(\mathbf{u}, \mathbf{v}) = \left(\sum_{j=1}^{d} |u_j - v_j|^q \right)^{1/q}.$$

$q = 1$: Manhattan. $q = 2$: Euclidean.

Mahalanobis, for a fixed $\mathbf{\Sigma} \succ 0$:

$$\delta_{\mathbf{\Sigma}}(\mathbf{u}, \mathbf{v}) = \sqrt{(\mathbf{u} - \mathbf{v})^\top \mathbf{\Sigma}^{-1} (\mathbf{u} - \mathbf{v})}.$$

Hamming, for categorical or binary coordinates:

$$\delta_H(\mathbf{u}, \mathbf{v}) = \sum_{j=1}^{d} \mathbf{1}_{\{u_j \neq v_j\}}.$$

The cluster representative minimizes the total within-cluster dissimilarity: mean for squared Euclidean loss ; coordinatewise median for Manhattan distance.

Scaling changes the geometry

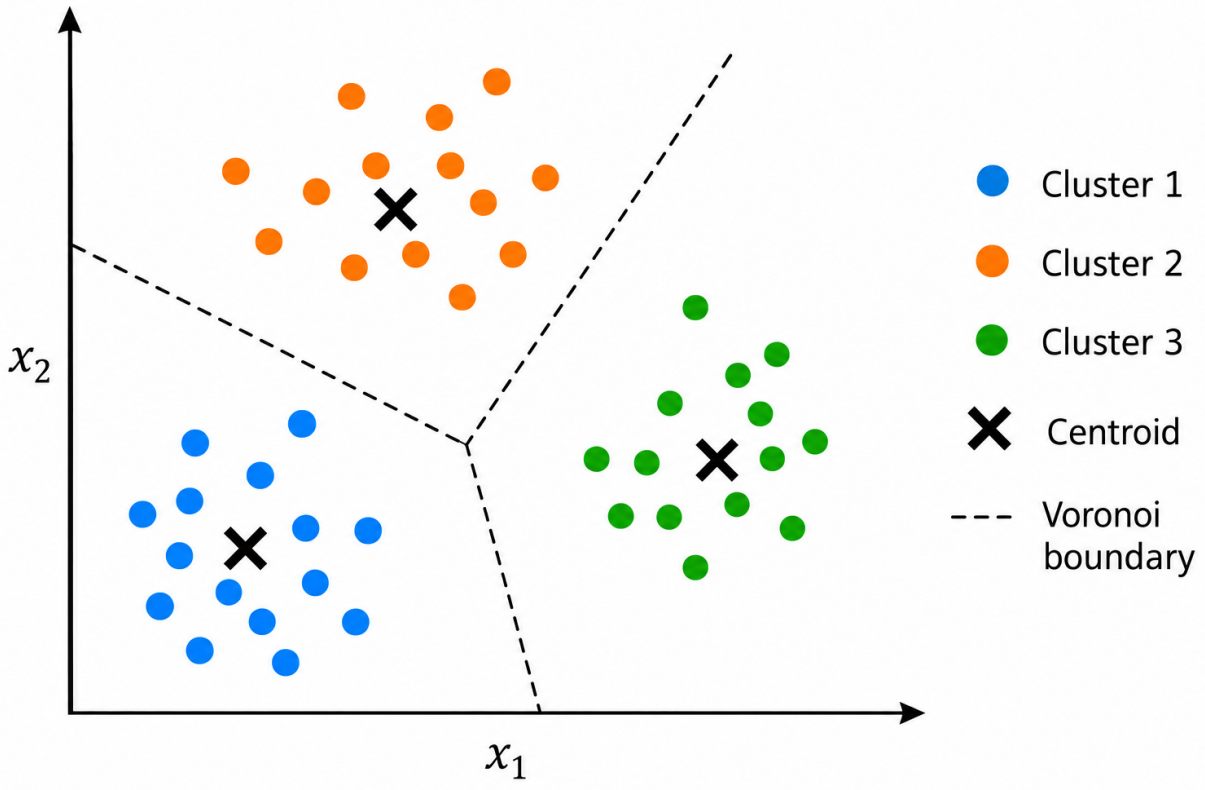
For positive feature scales s_j, Euclidean distance after scaling is

$$\delta(\mathbf{u}, \mathbf{v})^2 = \sum_{j=1}^{d} \frac{(u_j - v_j)^2}{s_j^2}.$$

- Features with larger numerical ranges can dominate Euclidean distances.
- Standardization changes the relative contributions of features to the distance, and may therefore affect the clustering result.
- Correlations may motivate whitening or a Mahalanobis distance.

Feature scaling affects pairwise distances and can significantly change the resulting clusters.

K-means: overview



Each point is assigned to the nearest centroid. Centroids are chosen to minimize the within-cluster sum of squared errors.

K-means: the criterion

Choose $1 \leq K \leq n$. Estimate the centroids $\boldsymbol{\mu} = (\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K)$ and the cluster assignments $\mathbf{z} = (z_1, \ldots, z_n)$:

$$\min_{\substack{z_i \in \{1,\ldots,K\} \\ \boldsymbol{\mu}_k \in \mathbb{R}^d}} F(\mathbf{z}, \boldsymbol{\mu}), \quad \text{with} \quad F(\mathbf{z}, \boldsymbol{\mu}) = \sum_{i=1}^{n} \|\mathbf{x}_i - \boldsymbol{\mu}_{z_i}\|_2^2.$$

Here $z_i = k$ means that observation $\mathbf{x}_i$ belongs to cluster k.

Every observation has equal weight. The objective is the total within-cluster sum of squares.

Non-convex optimization problem solved with an alternating minimization approach.

Assignment step: independent discrete problems

Fix the centroids $\boldsymbol{\mu}^{(t)}$. Each observation can be treated separately:

$$\boxed{z_i^{(t+1)} \in \arg\min_{1 \le k \le K} \|\mathbf{x}_i - \boldsymbol{\mu}_k^{(t)}\|_2^2.}$$

- Assign $\mathbf{x}_i$ to the nearest centroid.
- In case of equal distances, keep the previous assignment when possible.
- Define $C_k^{(t+1)} = \{i : z_i^{(t+1)} = k\}$.

For fixed centroids, assigning each observation to its nearest centroid globally minimizes F over the cluster assignments.

Centroid step: least-squares problems

Fix $\mathbf{z}^{(t+1)}$. For a nonempty cluster $C_k = C_k^{(t+1)}$,

$$g_k(\mathbf{u}) = \sum_{i \in C_k} \|\mathbf{x}_i - \mathbf{u}\|_2^2, \qquad n_k = |C_k|.$$

Centroid step: least-squares problems

Fix $\mathbf{z}^{(t+1)}$. For a nonempty cluster $C_k = C_k^{(t+1)}$,

$$g_k(\mathbf{u}) = \sum_{i \in C_k} \|\mathbf{x}_i - \mathbf{u}\|_2^2, \qquad n_k = |C_k|.$$

The exact minimizer is the empirical mean:

$$\boxed{\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{n_k} \sum_{i \in C_k} \mathbf{x}_i.}$$

Each centroid solves an independent least-squares problem. If a cluster is empty, its previous centroid can be retained.

Lloyd's algorithm

Initialization: choose K initial centroids $\boldsymbol{\mu}^{(0)}$.

1. Assign each observation to a nearest current centroid.
2. Replace each nonempty cluster's centroid by its mean.
3. Stop when assignments are unchanged, or when a practical tolerance or iteration limit is reached.

In practice: Initialize centroids with K-means++, run K-means several times, and keep the solution with the lowest objective value.

Course I: a difficult joint problem becomes two simple exact block minimizations. Initialization matters.

Why the objective decreases

The assignment step minimizes F with $\boldsymbol{\mu}$ fixed:

$$F(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}) \leq F(\mathbf{z}^{(t)}, \boldsymbol{\mu}^{(t)}).$$

The centroid step minimizes F with $\mathbf{z}$ fixed:

$$\boxed{F(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t+1)}) \leq F(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}) \leq F(\mathbf{z}^{(t)}, \boldsymbol{\mu}^{(t)}).}$$

Why the objective decreases

The assignment step minimizes F with $\boldsymbol{\mu}$ fixed:

$$F(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}) \leq F(\mathbf{z}^{(t)}, \boldsymbol{\mu}^{(t)}).$$

The centroid step minimizes F with $\mathbf{z}$ fixed:

$$\boxed{F(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t+1)}) \leq F(\mathbf{z}^{(t+1)}, \boldsymbol{\mu}^{(t)}) \leq F(\mathbf{z}^{(t)}, \boldsymbol{\mu}^{(t)}).}$$

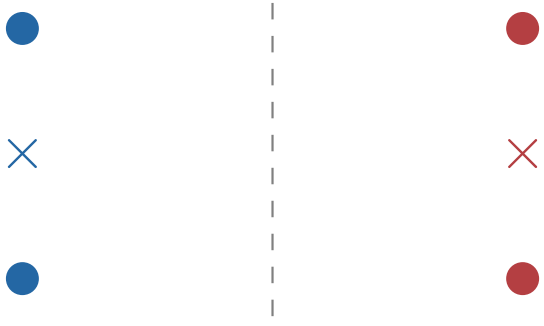
- The objective values converge because they decrease and are bounded below by zero.

At a fixed point, neither block update improves F. This blockwise optimality does not imply global optimality, since the joint problem is nonconvex.

Two fixed points, two different objective values

Four observations: $(0,0), (0,2), (4,0), (4,2)$; choose $K = 2$.

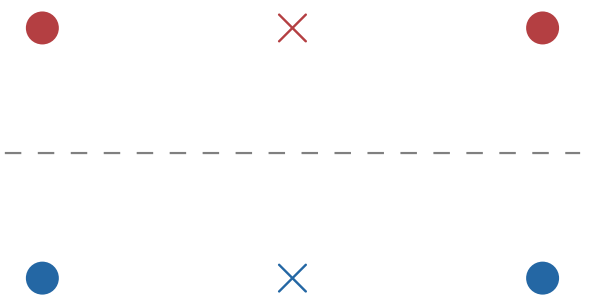
Split by horizontal position



$$\boldsymbol{\mu}_1 = (0,1), \quad \boldsymbol{\mu}_2 = (4,1)$$

$$F = 4.$$

Split by vertical position



$$\boldsymbol{\mu}_1 = (2,0), \quad \boldsymbol{\mu}_2 = (2,2)$$

$$F = 16.$$

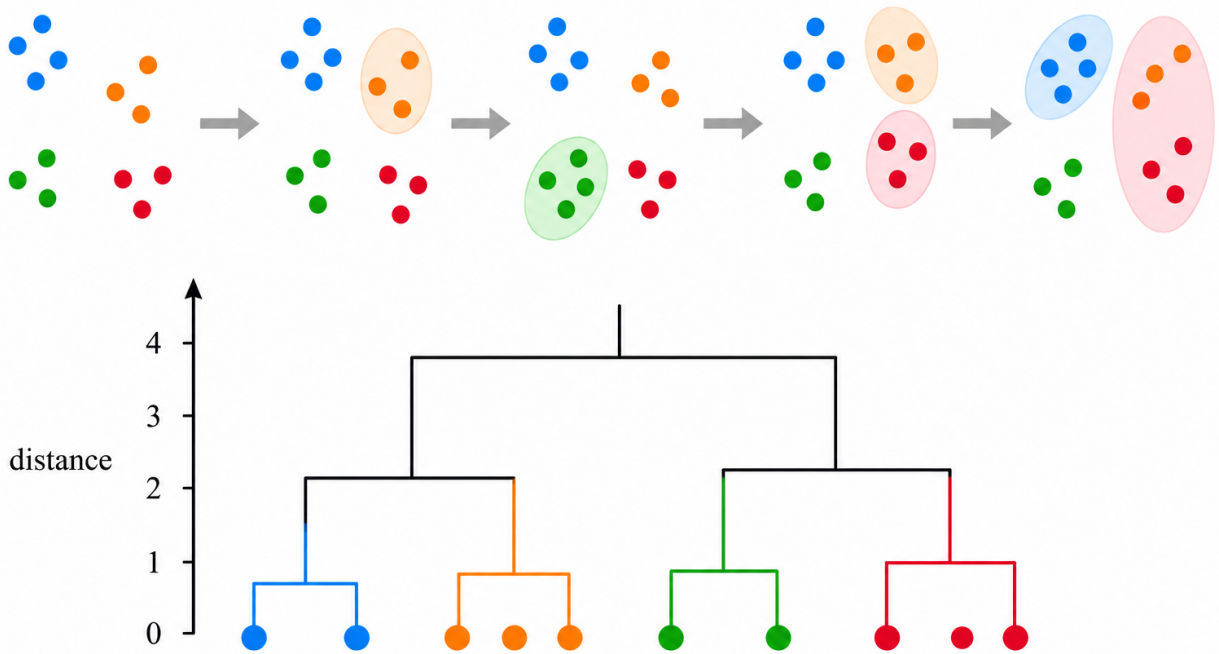
Both partitions are fixed points of Lloyd's algorithm, but have different objective values. The final solution depends on initialization and need not be globally optimal.

Geometry and limitations of K-means

- Squared loss gives large influence to distant observations.
- Every observation receives a label, including outliers.
- Nearest-centroid cells may split curved or intertwined groups.
- Unequal spreads and sizes can produce unintuitive partitions.

Improving optimization cannot repair a mismatch between the geometry and the desired groups.

Hierarchical clustering: overview



Agglomerative clustering starts with individual observations and progressively merges the closest clusters. The dendrogram records these merges; cutting it at different heights produces different partitions.

Agglomerative hierarchical clustering

Agglomerative construction

Start with n clusters, each containing one observation. At each step, merge the two closest clusters according to a chosen linkage criterion. A dendrogram represents the resulting hierarchy:

- Leaves correspond to individual observations.
- Internal nodes represent cluster merges.
- The height of each merge indicates its linkage value.

The number of clusters K need not be specified in advance. A partition into K clusters can be obtained by cutting the dendrogram.

Observation distance versus cluster linkage

The dissimilarity $\delta(\mathbf{x}_i, \mathbf{x}_{i'})$ compares two observations i and i'.
A linkage $L(A, B)$ compares two nonempty clusters of indices.

$$L_{\text{single}}(A, B) = \min_{i \in A, i' \in B} \delta(\mathbf{x}_i, \mathbf{x}_{i'}),$$

$$L_{\text{complete}}(A, B) = \max_{i \in A, i' \in B} \delta(\mathbf{x}_i, \mathbf{x}_{i'}),$$

$$L_{\text{average}}(A, B) = \frac{1}{|A||B|} \sum_{i \in A} \sum_{i' \in B} \delta(\mathbf{x}_i, \mathbf{x}_{i'}).$$

Different linkage criteria can produce different hierarchies from the same pairwise distances.

Linkage criteria interpretation



A linkage criterion defines the dissimilarity between two clusters: the minimum (single linkage), maximum (complete linkage), or average (average linkage) of pairwise distances.

Agglomerative algorithm

Input: observations (or pairwise dissimilarities), and a linkage L.

1. Initialize the active clusters as $\{1\}, \ldots, \{n\}$.
2. Select a distinct active pair (A, B) with smallest $L(A, B)$.
3. Record the merge and its height $h = L(A, B)$.
4. Replace A, B by $A \cup B$, and update its linkages.
5. Repeat until one cluster remains: exactly $n - 1$ merges.

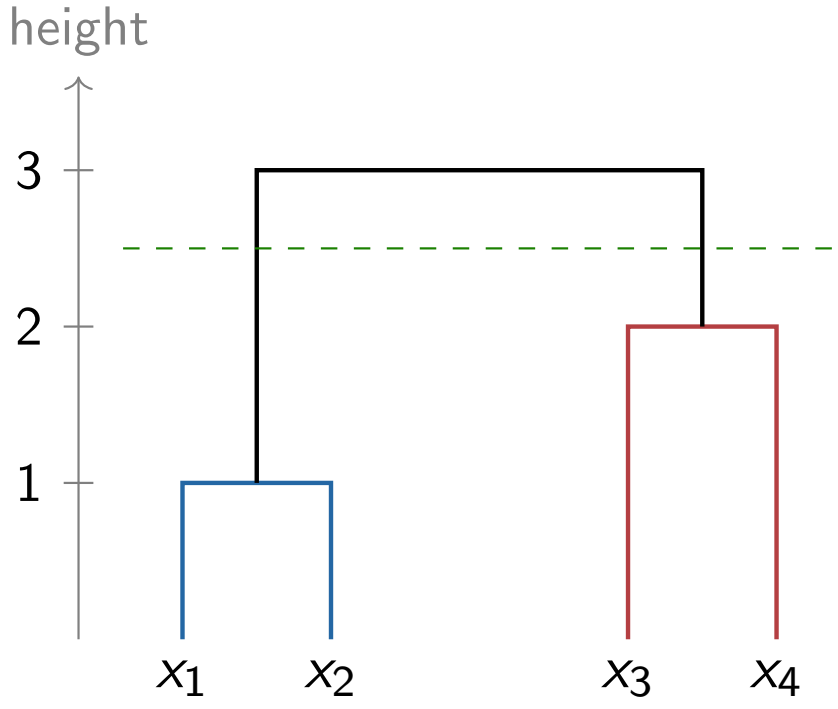
Agglomerative algorithm

Input: observations (or pairwise dissimilarities), and a linkage L.

1. Initialize the active clusters as $\{1\}, \ldots, \{n\}$.
2. Select a distinct active pair (A, B) with smallest $L(A, B)$.
3. Record the merge and its height $h = L(A, B)$.
4. Replace A, B by $A \cup B$, and update its linkages.
5. Repeat until one cluster remains: exactly $n - 1$ merges.

Agglomerative clustering is a greedy algorithm: once two clusters are merged, they cannot be separated. The resulting partition is not necessarily optimal.

Reading and cutting a dendrogram



For $2 \leq h < 3$, retain merges with height at most h:

$$\{1, 2\}, \qquad \{3, 4\}.$$

Two clusters remain.

Alternatively, stop after $n - K$ merges to obtain K clusters.

Only the vertical axis represents linkage values; the horizontal positions of observations have no distance interpretation.

Single linkage and connected components

At threshold h, connect i and i' whenever $\delta(\mathbf{x}_i, \mathbf{x}'_i) \leq h$.

The single-linkage clusters are the connected components.



Endpoints may be much farther apart than h.

Single linkage and connected components

At threshold h, connect i and i' whenever $\delta(\mathbf{x}_i, \mathbf{x}'_i) \leq h$.

The single-linkage clusters are the connected components.

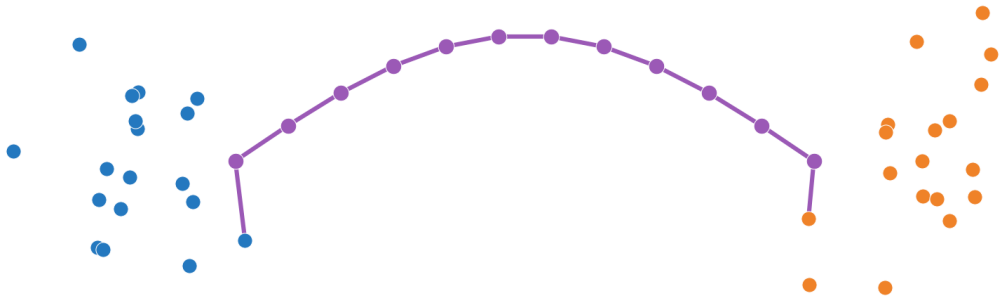


Endpoints may be much farther apart than h.

- A chain can recover an elongated or curved group.
- A sparse bridge can also merge two otherwise distinct groups.

This chaining effect motivates density-based methods.

Single linkage chaining effect



Single linkage connects observations through chains of short distances. This allows elongated clusters to be recovered, but sparse bridges may also connect otherwise distinct groups.

Ward's method: a connection with K-means

Let $\boldsymbol{\mu}_A = |A|^{-1} \sum_{i \in A} \mathbf{x}_i$ and $W(A) = \sum_{i \in A} \|\mathbf{x}_i - \boldsymbol{\mu}_A\|_2^2$.

$$\Delta(A, B) = W(A \cup B) - W(A) - W(B)$$

$$= \boxed{\frac{|A||B|}{|A| + |B|} \|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|_2^2.}$$

Ward's rule: merge the pair with smallest $\Delta(A, B)$.

Each merge produces the smallest available increase in within-cluster inertia.

Ward uses the same squared Euclidean geometry as K-means. Its greedy hierarchy is not the global K-means solution.

Ward's method: visual interpretation



Ward's method merges the pair of clusters producing the smallest increase in within-cluster squared error. It uses the same criterion as K-means, but with irreversible, successive merges.

DBSCAN: clustering based on local density

DBSCAN uses local neighbourhood counts

$$N_\varepsilon(i) = \{i' : \delta(\mathbf{x}_i, \mathbf{x}'_i) \leq \varepsilon\}, \qquad \varepsilon > 0.$$

Choose a minimum count m (`min_samples` / MinPts).

Convention here: i itself belongs to $N_\varepsilon(i)$.

DBSCAN: clustering based on local density

DBSCAN uses local neighbourhood counts

$$N_\varepsilon(i) = \{i' : \delta(\mathbf{x}_i, \mathbf{x}_i') \leq \varepsilon\}, \qquad \varepsilon > 0.$$

Choose a minimum count m (`min_samples` / MinPts).

Convention here: i itself belongs to $N_\varepsilon(i)$.

- A core point satisfies $|N_\varepsilon(i)| \geq m$.
- Connect nearby core points into dense groups.
- Allow sparse observations to remain unassigned.

Unlike K-means, DBSCAN does not require a predefined number of clusters. The number of clusters is determined by the data and the parameters ε and m.

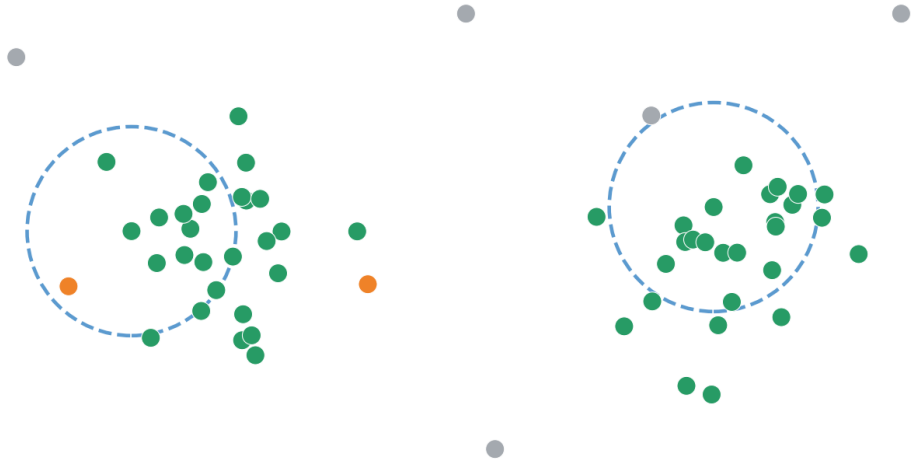
Core, border and noise points

DBSCAN distinguishes three types of observations according to their local neighbourhood:

- **Core point:** its ε-neighbourhood contains at least m observations (including itself).
- **Border point:** it is not a core point, but lies within distance ε of a core point.
- **Noise point:** it is neither a core point nor a border point.

The classification depends on ε and m: changing these parameters can change which observations are considered core, border or noise.

Core, border and noise points



Core points have at least m observations in their ε-neighbourhood.
Border points are within distance ε of a core point, but are not core themselves.
Noise points are neither core nor border.
Dashed circles illustrate ε-neighbourhoods.

DBSCAN as a graph construction

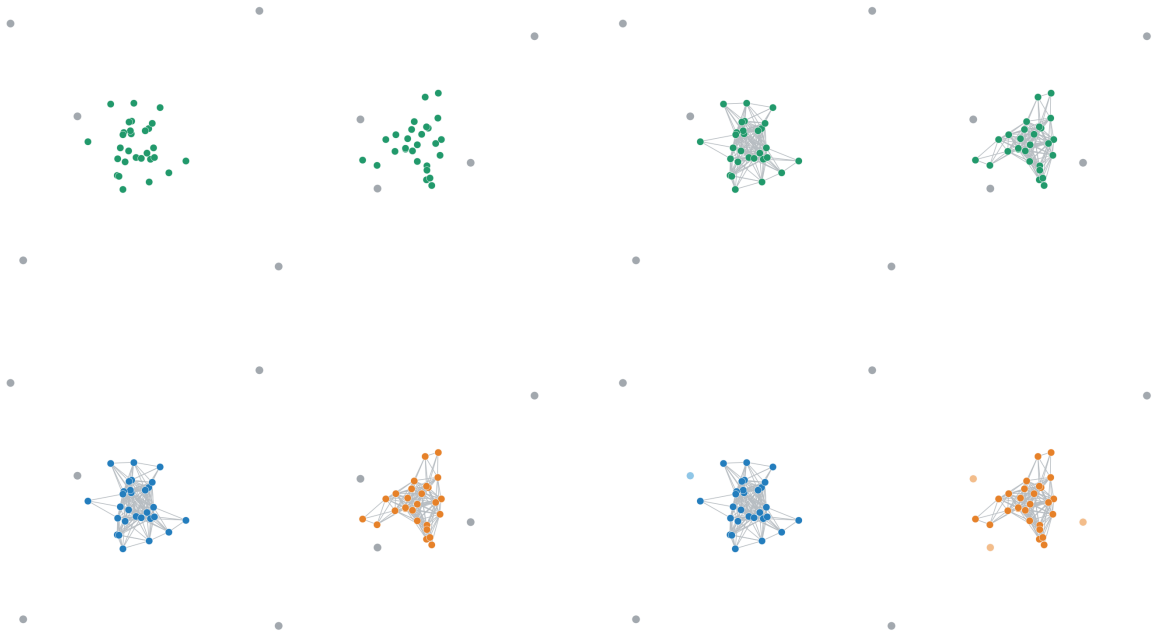
1. Identify all core observations.
2. Build a graph on the core observations, with an edge when their distance is at most ε.
3. Find its connected components.
4. Assign each border point to a cluster containing at least one core point within distance ε.
5. Label the remaining observations as noise.

DBSCAN as a graph construction

1. Identify all core observations.
2. Build a graph on the core observations, with an edge when their distance is at most ε.
3. Find its connected components.
4. Assign each border point to a cluster containing at least one core point within distance ε.
5. Label the remaining observations as noise.

Border points do not connect two core components. If a border point touches several components, its assignment can depend on traversal order.

DBSCAN principle



DBSCAN identifies core points, connects neighbouring core points, and finds the connected components. Border points are then attached to adjacent components; other observations remain unassigned.

How the parameters change the result

Increasing ε at fixed m

Neighbourhoods grow; more points become core and core components can merge.

Increasing m at fixed ε

Fewer points are core; groups may split or disappear and more points may be noise.

How the parameters change the result

Increasing ε at fixed m

Neighbourhoods grow; more points become core and core components can merge.

Increasing m at fixed ε

Fewer points are core; groups may split or disappear and more points may be noise.

The difficulty with varying densities

A radius large enough to connect a sparse group may already merge two dense groups.

DBSCAN uses a single distance scale. Its parameters must be interpreted together with feature scaling.

What DBSCAN does and does not guarantee

Useful properties

Chains of nearby core points can follow curved groups. Observations outside core neighbourhoods remain noise. No prescribed K is needed.

Limitations

- Nearby groups can merge through dense bridges.
- Strongly varying densities challenge a single radius.
- In high dimensions, neighbourhoods may be less informative.

Noise handling is not universal robustness. The method still depends on its distance and density scale.

HDBSCAN main idea



HDBSCAN builds a hierarchy of density-based groupings instead of using a single neighbourhood radius. It selects stable clusters, potentially at different density levels.

From one density scale to a hierarchy

DBSCAN uses one global density scale ε.

HDBSCAN in five stages

1. Compute density-aware pairwise edge weights.
2. Build a minimum spanning tree (MST).
3. Form a hierarchy by thresholding its edges.
4. Condense the hierarchy using a minimum cluster size.
5. Select persistent clusters from the condensed tree.

A flat clustering can combine branches selected at different density levels.
This replaces choosing one global ε.

Mutual reachability and sparse neighbourhoods

Let $c_m(i)$ be the smallest radius containing at least m observations around $\mathbf{x}_i$, including $\mathbf{x}_i$ itself. For distinct observations define the edge weight

$$\boxed{w_{i,i'} = \max\{c_m(i), c_m(i'), \delta(\mathbf{x}_i, \mathbf{x}_{i'})\}.}$$

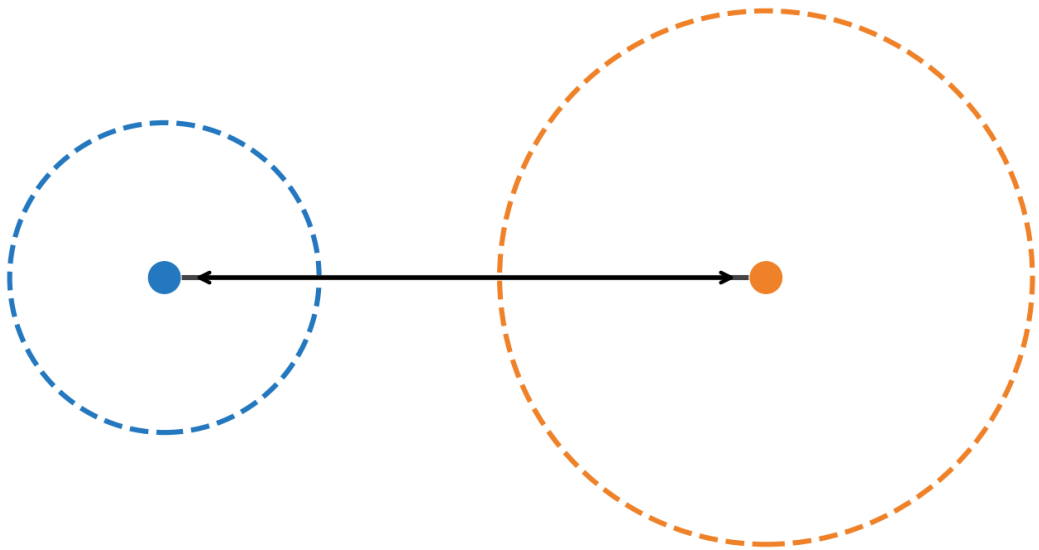
- Both points must have sufficiently dense neighbourhoods.
- They must also be close enough to each other.

For example,

$$c_m(i) = 0.2, \quad c_m(i') = 0.8, \quad \delta(\mathbf{x}_i, \mathbf{x}_{i'}) = 0.3 \quad \Longrightarrow \quad w_{i,i'} = 0.8.$$

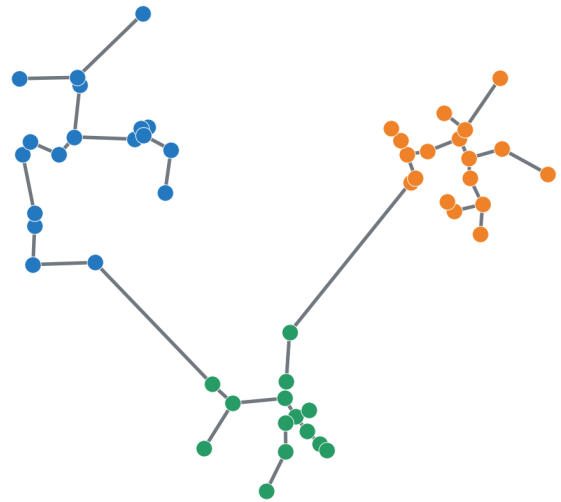
This modifies edge weights; no explicit movement of the observations is required.

Mutual reachability



Mutual reachability combines the distance between two observations with their core distances $w_{i,i'} = \max\{c_m(i), c_m(i'), \delta(\mathbf{x}_i, \mathbf{x}_{i'})\}$. This increases the effective distance of connections involving sparse neighbourhoods.

Minimum spanning tree



Thresholding mutual-reachability weights produces a hierarchy of connected components. A minimum spanning tree, which connects all observations using $n - 1$ edges of minimum total weight, provides an efficient representation of this hierarchy.

HDBSCAN selects sufficiently large, stable clusters from the density hierarchy, without requiring a single global neighbourhood radius.

Four methods, four constructions

Method	Criterion / construction	Main choices
K-means	Squared distortion; alternating minimization	K, scaling, initialization
Agglomerative	Greedy merges; nested partitions	Dissimilarity, linkage, cut
DBSCAN	Core connectivity and border attachment	Dissimilarity, ε, neighbour count
HDBSCAN	Density hierarchy and stable branch selection	Dissimilarity, neighbour count, minimum size

All impose modeling choices. Only some begin with a global data-fitting objective.

Take-home messages

1. Representation and dissimilarity determine what “close” means.
2. K-means decreases a nonconvex objective by exact block updates.
3. Hierarchical clustering keeps nested resolutions; linkage and cut both matter.
4. DBSCAN connects dense neighbourhoods and can reject noise.
5. HDBSCAN builds a density hierarchy and selects persistent branches.

Always separate the chosen model, the computed result and its interpretation.